

БИЕ ДААЛТ 14

Integration & API Testing — Postman + Newman

Үндсэн мэдээлэл

- **Сэдэв:** API Testing — Postman collection + Newman CI (Лекц 14)
- **Төрөл:** Hybrid лаб — анги дотор эхэлж, гэрт дуусгана. 4 хэсэгтэй (А = Setup, Б = Collection, В = Tests, Г = CI)
- **Хугацаа:** 1 долоо хоног
- **Үнэлгээ:** Pass / Fail (доорх шалгуурын дагуу)
- **Хэрэгсэл:** Postman (Desktop, үнэгүй), Node.js + Newman, Git + GitHub. Сонгосон API-аас хамаарна (curl, hosting шаардлагагүй).
- **Хүлээлгэх хэлбэр:** GitHub repository-ийн линкийг Microsoft Teams-р илгээнэ (доор дурдсан журмаар).

Зорилго

Лекц 14-д үзсэн integration & API testing-ийн зарчмыг бодит API дээр практикжуулах. Энэ лабораторийн төгсгөлд оюутан:

- Postman-аар collection, environment, variable-ийг зохион байгуулж сурах
- Pre-request ба Test script-аар status, schema, latency, business rule-ийг баталгаажуулах
- Request-үүдийг chain хийж, нэг response-оос дараагийн request-д зориулсан хувьсагч авах
- Newman CLI ашиглан collection-ыг командын мөрнөөс ажиллуулж, CI pipeline-д суулгах
- Тестийн пирамидын дунд давхаргад API testing хаана зохистой ордогийг ялгах

Сэдвийн сонголт — Аль API-г тестлэх вэ?

Дараах 3 хувилбараас НЭГИЙГ сонгож, эхний 30 минутад багштай зөвшилцөнө. Сонгосон API-д GET-ээс гадна нэг бичих үйлдэл (POST/PUT/DELETE) заавал байна.

Хувилбар 1 — Public API (хамгийн хялбар)

Бэлэн нээлттэй API-аас сонгож, тест бичнэ. Auth шаардахгүй эсвэл token-той хувилбар сонгож болно.

- JSONPlaceholder — jsonplaceholder.typicode.com (CRUD хуурамч endpoint, auth байхгүй)
- Reqres — reqres.in (registration, login, paginated list — pagination тест боломжтой)
- PokeAPI — pokeapi.co (read-only, query/path параметрийн жишээ)

- OpenWeatherMap — api token шаардана (environment variable practice)
- DummyJSON — dummyjson.com (CRUD, login + JWT — chain хийх дасгал)

Хувилбар 2 — Бие даалт 11-ийн API-аа дахин ашиглах

Бие даалт 11-д бичсэн өөрийн REST API-аа local-д ажиллуулж, түүн дээр тест бичнэ. Хамгийн их value авах хувилбар — өөрийн design-ын дутагдлыг тест нь илрүүлнэ.

Хувилбар 3 — Жижиг өөрийн Express/FastAPI server

≤ 50 мөр код бүхий жижиг сервер бичин ажиллуулна. Зорилго — "тестийн төлөө" сервер биш, тестлэх боломжтой жижиг бодит сервер.

- ≥ 4 endpoint (нэг нь POST/PUT/DELETE байх)
- ≥ 1 endpoint нь body validation хийнэ (400 буцаах)
- ≥ 1 endpoint нь "not found" буюу 404 буцаах

Хэсгүүд (4)

А ба Б хэсгийг анги дотор эхлүүлнэ. В ба Г хэсгийг гэрт дуусгана.

А хэсэг — Setup (Анги дотор, ≈ 30 мин)

Гарааны setup. Энэ хэсгийн төгсгөлд Postman нээгдсэн, эхний request амжилттай явсан байна.

Хийх ажил

- Postman Desktop App татаж суулгах (postman.com/downloads). Бүртгэл үүсгэх (sign-in заавал биш ч хадгалах боломжтой болохын тулд хийе).
- Сонгосон API-гаа сонгож, документацыг 5 минут уншсан байх — endpoint, auth, rate limit
- Шинэ Workspace үүсгэх: "F.CSM311 — Lab14"
- Шинэ Collection үүсгэх: "<Овог> — <API name>"
- 3+ Environment үүсгэх: dev, staging, prod (зөвхөн dev-ийг бөглөнө). Бүгдэд `baseUrl` хувьсагч
- 1 GET request бичин Send хийж 200 эсвэл хүчинтэй status авсан байх

Гаргах файлууд (А хэсгээс)

partA/SETUP.md	– сонгосон API, түүний brief, auth, base URL
partA/screenshot.png	– эхний амжилттай request-ийн screenshot

Б хэсэг — Postman Collection бичих (Анги дотор + Гэрт)

Сонгосон API-аасаа дор хаяж 8 request-ийг зохион байгуулна. Тестээс өмнө collection-ыг "document" мэт ашиглахуйцаар цэгцлэх нь чухал.

Шаардлагатай request-ийн төрөл (бүгд байх ёстой)

Доорх 6 төрөл бүрээс ≥ 1 request байх ёстой. Нийт ≥ 8 request.

Төрөл	Юу шалгана	Жишээ
Happy GET	200, body байгаа	GET /users (list)
GET by id	200 эсвэл 404	GET /users/1
POST create	201, шинэ id буцаах	POST /users
PUT/PATCH	200, өөрчлөлт хадгалагдсан	PUT /users/1
DELETE	204 эсвэл 200	DELETE /users/1
Error case	4xx (400/401/404)	GET /users/999999

Зохион байгуулалтын шаардлага

- Folder ашиглан request-уудыг бүлэглэх (жишээ нь: "Users", "Auth", "Errors")
- Бүх URL `{{baseUrl}}` хувьсагчтай — hardcode хийсэн URL fail-д хүргэнэ
- Auth шаардсан endpoint-д Authorization header-ийг collection-level дээр тохируулах (request-бүрд давтахгүй)
- Body нь JSON-той бол `application/json` content-type автомат тохируулагдсан байх
- Бүх request-д товч description (1 мөр) бичсэн байх

Жишээ: chained request

Нэг request-ийн response-оос дараагийнх нь хэрэглэх хувьсагч авах нь Postman-ы зүрхэн боломж. Login → Token → Get profile-ийн жишээ:

```
// Login request → Tests tab
const json = pm.response.json();
pm.environment.set("token", json.token);
pm.environment.set("userId", json.id);
// Get profile request → Authorization
// Header: Authorization: Bearer {{token}}
// URL: {{baseUrl}}/users/{{userId}}
```

В хэсэг — Test script бичих (Гэрт)

Postman-ы хамгийн чухал хэсэг — Tests tab-д JS код бичин баталгаажуулалт нэмэх. Pass/Fail тоо нь Newman report-д харагдана.

Шаардлагатай assertion-ы төрөл (≥ 5 төрөл, нийт ≥ 15 тест)

Assertion төрөл	Жишээ
Status code	pm.response.to.have.status(200)
Response time	pm.expect(pm.response.responseTime).to.be.below(1000)
JSON schema / property	pm.expect(d).to.have.property("email")
Утгын тип	pm.expect(d.id).to.be.a("number")
Бизнес дүрэм	pm.expect(d.email).to.match(/@/)
Header	pm.response.to.have.header("Content-Type")

Жишээ: бүрэн test script

```
pm.test("Status 200", () => {
  pm.response.to.have.status(200);
});
pm.test("Response < 1s", () => {
  pm.expect(pm.response.responseTime).to.be.below(1000);
});
pm.test("Schema valid", () => {
  const d = pm.response.json();
  pm.expect(d).to.have.property("id");
  pm.expect(d).to.have.property("email");
  pm.expect(d.email).to.be.a("string").and.match(/@/);
});
pm.test("Content-Type = JSON", () => {
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});
```

Заавал үзэх 2 кейс

- **Negative test:** Буруу payload эсвэл байхгүй id явуулж 400/404 эргэж ирэхийг шалгах. "Хэвийн ажиллах"-ыг шалгах хангалтгүй — алдааны зам ч бас contract.
- **Pre-request script ≥ 1:** Token авах, timestamp оруулах, эсвэл random data үүсгэх. Энэ нь test-ийн "Arrange" хэсгийг автоматжуулна.

Г хэсэг — Newman + GitHub Actions (Гэрт)

Postman UI-аас гадна командын мөрнөөс ажиллуулж, CI-д суулгана. Энэ хэсэг бол лабораторийн "production-ready" хэсэг.

Newman local-д ажиллуулах

- Postman-аас collection-ыг export хийх (v2.1 формат): postman/collection.json
- Environment-ыг export: postman/env.dev.json (token зэрэг secret-ыг placeholder болгох — REAL_TOKEN_HERE)
- Newman суулгах: `npm install -g newman`
- Local-д ажиллуулах:

```
newman run postman/collection.json -e postman/env.dev.json
```

- HTML report үүсгэх:

```
npm install -g newman newman-reporter-htmlextra
```

```
newman run postman/collection.json \
-e postman/env.dev.json \
--reporters cli,htmlextra \
--reporter-htmlextra-export reports/api.html
```

GitHub Actions workflow

Репозиторий-д `.github/workflows/api-tests.yml` файл үүсгэн дор хаяж push дээр Newman ажиллах CI бүтээх:

```
# Workflow-ийн нэр
name: API tests

# Хэзээ ажиллах нөхцөл
on: [push, pull_request]

jobs:
  newman:
    # Ажиллах орчин
    runs-on: ubuntu-latest

    steps:
      # 1. Кодыг татаж авах
      - name: Checkout repository
        uses: actions/checkout@v4

      # 2. Node.js суулгах
      - name: Install Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      # 3. Newman болон нэмэлт хэрэгслүүдийг суулгах
      - name: Install Newman
        run: npm install -g newman newman-reporter-htmlextra

      # 4. Newman тестүүдийг ажиллуулах
      - name: Run API tests
        run: |
          newman run postman/collection.json \
            -e postman/env.ci.json \
            --reporters cli,junit,htmlextra \
            --reporter-junit-export reports/api.xml \
            --reporter-htmlextra-export reports/api.html

      # 5. Тестийн тайланг файл хэлбэрээр хадгалах
      - name: Upload Artifact
        uses: actions/upload-artifact@v4
        if: always() # Тест амжилтгүй болсон ч тайланг татаж авах боломжтой байлгах
        with:
          name: api-test-report
          path: reports/
```

- Workflow-ийн ≥ 1 run амжилттай ногоон болсон байх (Actions tab-д харагдана)
- Secret token хэрэгтэй бол GitHub Secrets ашиглан `env.ci.json`-д placeholder substitute хийх

Гаргах файлууд (Г хэсгээс)

└─ .github/	
└─ workflows/	
└─ api-tests.yml	# GitHub Actions тохиргоо
└─ postman/	
└─ collection.json	# Экспорт хийсэн collection
└─ env.dev.json	# Хөгжүүлэлтийн орчин (Secrets хасагдсан)
└─ env.ci.json	# CI орчин (Placeholder ашиглах)
└─ reports/	
└─ api.html	# Эхний амжилттай run-ы тайлан
└─ README.md	# Хэрхэн ажиллуулах заавар

Repository-ийн бүтэц

Доорх бүтцийг яг дагана:

bie-daalt-14/	
└─ .github/	
└─ workflows/	
└─ api-tests.yml	# CI/CD (GitHub Actions)
└─ partA/	
└─ SETUP.md	# API сонголт болон тайлбар
└─ screenshot.png	# Эхний амжилттай request-ийн зураг
└─ postman/	
└─ collection.json	# ≥ 8 request (folder-р бүлэглэсэн)
└─ env.dev.json	# Dev environment (Secret placeholder)
└─ env.ci.json	# CI environment (Secret placeholder)
└─ reports/	
└─ api.html	# Newman-ийн HTML тайлан
└─ server/	
└─ index.js	# Local server (Хувилбар 3 бол)
└─ README.md	# API тайлбар, ажиллуулах заавар
└─ REFLECTION.md	# 5 асуултын хариулт

Эргэцүүлэл — REFLECTION.md (≈ 400 үг)

Хувилбараас үл хамаарч дараах 5 асуултанд богино хариулна:

- Аль assertion (тест) хамгийн их үнэ цэнэтэй санагдсан вэ? Яагаад?
- Negative test 1+ жишээгээ дэлгэрэнгүй тайлбарла — яг ямар алдааг олох вэ?
- Postman дотор амжилттай ажиллаж байсан тест Newman-д орвол fail болсон уу? Яагаад?
- Token эсвэл secret-ыг хэрхэн зохицуулсан вэ? (env-ийн соёл)
- API өөрчлөгдвөл collection-ийн аль хэсэг ХАМГИЙН ИХ эвдрэх вэ? Үүнийг тестийн "эмзэг газар" гэж нэрлэдэг — яаж бууруулах вэ?

Pass / Fail шалгуур

Pass болох нөхцөл (бүгд биелсэн байх):

А хэсэг

- ☐ SETUP.md: сонгосон API, brief, base URL, auth байгаа
- ☐ Screenshot: эхний амжилттай 200 (эсвэл хүчинтэй status) request

Б хэсэг

- ☐ ≥ 8 request, доод тал нь 6 төрлийн request бүрд ≥ 1 (Happy GET, GET by id, POST, PUT/PATCH, DELETE, Error)
- ☐ Бүх URL {{baseUrl}}-ыг ашигласан (hardcode байхгүй)
- ☐ Folder ашиглан бүлэглэсэн
- ☐ ≥ 1 request нь өмнөх response-оос авсан хувьсагчийг ашигласан (chain)

В хэсэг

- ☐ ≥ 15 нийт тест assertion, ≥ 5 төрлийн assertion (status, latency, schema, type, business rule, header)
- ☐ ≥ 1 negative test (4xx шалгасан)
- ☐ ≥ 1 pre-request script ажиллаж байгаа

Г хэсэг

- ☐ Newman local-д амжилттай ажиллана (бүх тест pass)
- ☐ .github/workflows/api-tests.yml файл байгаа
- ☐ Дор хаяж 1 ногоон workflow run (Actions tab харуулна)
- ☐ HTML report артефакт болж upload хийгдсэн

Бусад

- ☐ REFLECTION.md (5 асуултанд хариулсан, ≈ 400 үг)
- ☐ README.md: clone хийгээд яаж ажиллуулахыг тайлбарласан
- ☐ ≥ 8 commit, ≥ 3 өөр өдөр (нэг өдрийн rush биш)

Fail болох нөхцөл (нэг л биелэхэд):

- Хэсэг бүхэлдээ дутуу (А, Б, В, Г-ийн аль нэг)
- Бүх ажил нэг өдрийн дотор хийгдсэн (Git history харуулна)
- Нэг том commit (code dump) — алхам алхамаар ажилласан гэж үнэмшихгүй
- Newman ажиллахгүй (collection.json эвдэрсэн, env.json дутуу)
- CI workflow ногоон болоогүй (бүх run улаан)
- Token/secret-ыг шууд commit хийсэн (Хувилбар 1-ийн token-той API сонгосон бол ялангуяа анхаарна)
- Хугацаа 5+ хоногоор хэтэрсэн

Шударга байдал ба AI ашиглалт

Лекц 13-д үзсэнчлэн AI ашиглах нь зөвшөөрөгдсөн, гэхдээ:

- AI үүсгэсэн test script-ийг өөрөө ойлгож, тайлбарлаж чаддаг байх — шалгалтад асууж болно
- Hallucination сэжигтэй: AI "pm.response.to.have.bodyJson(...)" мэтийн байхгүй API санал болгож болзошгүй — Postman документац дээр шалгах
- Token, API key-г commit хийхгүй — AI санамсаргүй санал болговол шууд татгалзах
- Хувилбар 2 (Бие даалт 11)-ийг сонгосон бол өмнөх ажилаа дахин ашиглах нь зөв — гэхдээ test нь шинээр бичигдсэн байх ёстой

Зөвлөгөө

- API сонгохдоо документ нь сайтай эсэхэд анхаар. Документгүй API нь хүндрүүлдэг.
- Token-той API сонгохдоо free tier байгаа эсэхийг 5 минутад шалга — rate limit-д өртвөл CI улаан болдог.
- Эхлээд 2-3 request бичээд 1 тест нэм — "бүгдийг бичээд дараа тест нэмэх" нь fail-ийн зам.
- Newman local-д ажилласан үед л CI-д push хий — CI улаан болохоос сэргийл.
- env.dev.json-доо REAL token бүү хадгал. Placeholder ашиглан, README-д "REPLACE_THIS" гэж бич.
- HTML report-ыг нээж үзэж, аль ч request-ийн "Console" хэсэгт алдаа байгаа эсэхийг шалга.
- Эргэлзэж байвал Postman "Public API Network"-ийн жишээ collection-ыг import хийж, бүтцийг ажигла.

Хүлээлгэх журам

- GitHub дээр шинэ public (эсвэл багшийг collaborator болгосон private) repository үүсгэ — нэр: bie-daalt-14
- Дээрх бүтцийг дагаж бүх ажлыг commit, push хий
- Сүүлийн commit нь хугацаанд багтсан байх (Teams илгээсэн хугацаа биш, GitHub дээрх commit-ийн хугацаа үнэлэгдэнэ)
- Microsoft Teams → F.CSM311 бар → Assignments → "Бие даалт 14"
- "Add work" товчоор GitHub repo-ийн линкийг оруулах
- "Turn in" товчийг ЗААВАЛ дар — turn in дараагүй бол ажил хүлээн авагдахгүй

— Амжилт хүсье! —